Analyzing Weather Patterns Affecting Crop Yield

Project report: Topic | Coding | Content

1. Project Title and Objective

Title: Analyzing Weather Patterns Affecting Crop Yield. **Objective:** find which weather variables drive yield (t/ha) and predict yield from them.

Coding: imports and setup

```
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

2. Dataset Description

600 rows, 9 columns: region, year, rainfall_mm, avg_temp_c, humidity_pct, sunshine_hrs, heat_stress_days, fertilizer_kg_ha, yield_t_ha (target). Simulated data; mean yield 5.01 t/ha. Replace with `pd.read_csv()` for real data.

Coding: create / load data

```
# 1. DATA (synthetic; replace with real CSV via pd.read_csv("crop_weather.csv"))
rng = np.random.default_rng(42)
n = 600
df = pd.DataFrame({
    "region": rng.choice(["Delta", "North", "South", "West", "Coastal"], n),
    "year": rng.integers(2005, 2025, n),
    "rainfall_mm": rng.normal(850, 220, n).clip(250, 1500),
    "avg_temp_c": rng.normal(27, 2.5, n),
    "humidity_pct": rng.normal(68, 9, n).clip(30, 95),
    "sunshine_hrs": rng.normal(6.5, 1.1, n).clip(3, 10),
    "heat_stress_days": rng.poisson(8, n),
    "fertilizer_kg_ha": rng.normal(120, 25, n).clip(40, 200),
})
y = (2.0
     - ((df.rainfall_mm - 900) / 400) ** 2 * 1.2
     - ((df.avg_temp_c - 26) / 3) ** 2 * 0.9
     + 0.25 * (df.sunshine_hrs - 6.5)
     - 0.07 * df.heat_stress_days
     + 0.012 * (df.fertilizer_kg_ha - 120)
     + 0.02 * (df.year - 2005)
     + rng.normal(0, 0.25, n))
df["yield_t_ha"] = (4.5 + y).clip(0.8, None).round(2)
# inject missing values to demo cleaning
for c in ["humidity_pct", "rainfall_mm"]:
    df.loc[rng.choice(n, 15, replace=False), c] = np.nan
df.to_csv("crop_weather.csv", index=False)
```

3. Approach and Techniques

Median imputation by region, scatter plots and correlations, then Linear Regression vs Random Forest (80/20 split, 5-fold CV).

Coding: data cleaning

```
# 2. CLEANING
missing_before = int(df.isna().sum().sum())
df[["humidity_pct", "rainfall_mm"]] = df[["humidity_pct", "rainfall_mm"]].fillna(
    df.groupby("region")[["humidity_pct", "rainfall_mm"]].transform("median"))
```

Coding: EDA and charts

```
# 3. EDA
features = ["rainfall_mm", "avg_temp_c", "humidity_pct", "sunshine_hrs",
            "heat_stress_days", "fertilizer_kg_ha", "year"]
corr = df[features + ["yield_t_ha"]].corr()["yield_t_ha"].drop("yield_t_ha").sort_values()

fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
ax[0].scatter(df.rainfall_mm, df.yield_t_ha, s=10, alpha=.5, color="#2e7d32")
ax[0].set(xlabel="Rainfall (mm)", ylabel="Yield (t/ha)", title="Rainfall vs Yield")
ax[1].scatter(df.avg_temp_c, df.yield_t_ha, s=10, alpha=.5, color="#ef6c00")
ax[1].set(xlabel="Avg temperature (°C)", title="Temperature vs Yield")
ax[2].scatter(df.heat_stress_days, df.yield_t_ha, s=10, alpha=.5, color="#c62828")
ax[2].set(xlabel="Heat-stress days", title="Heat Stress vs Yield")
plt.tight_layout(); plt.savefig("fig_scatter.png", dpi=150); plt.close()

plt.figure(figsize=(6.5, 4))
plt.barh(corr.index, corr.values, color=["#c62828" if v < 0 else "#2e7d32" for v in corr.values])
plt.xlabel("Pearson correlation with yield"); plt.title("Correlation of Features with Yield")
plt.tight_layout(); plt.savefig("fig_corr.png", dpi=150); plt.close()
```

Coding: models and evaluation

```
# 4. MODELS
X, t = df[features], df["yield_t_ha"]
Xtr, Xte, ytr, yte = train_test_split(X, t, test_size=0.2, random_state=42)
lin = LinearRegression().fit(Xtr, ytr)
rf = RandomForestRegressor(n_estimators=300, random_state=42).fit(Xtr, ytr)
res = {}
for name, m in [("Linear Regression", lin), ("Random Forest", rf)]:
    p = m.predict(Xte)
    res[name] = {"R2": r2_score(yte, p), "MAE": mean_absolute_error(yte, p),
                 "RMSE": float(np.sqrt(mean_squared_error(yte, p))),
                 "CV_R2": cross_val_score(m, X, t, cv=5, scoring="r2").mean()}
imp = pd.Series(rf.feature_importances_, index=features).sort_values()

plt.figure(figsize=(6.5, 4))
plt.barh(imp.index, imp.values, color="#1565c0")
plt.xlabel("Importance"); plt.title("Random Forest Feature Importance")
plt.tight_layout(); plt.savefig("fig_importance.png", dpi=150); plt.close()

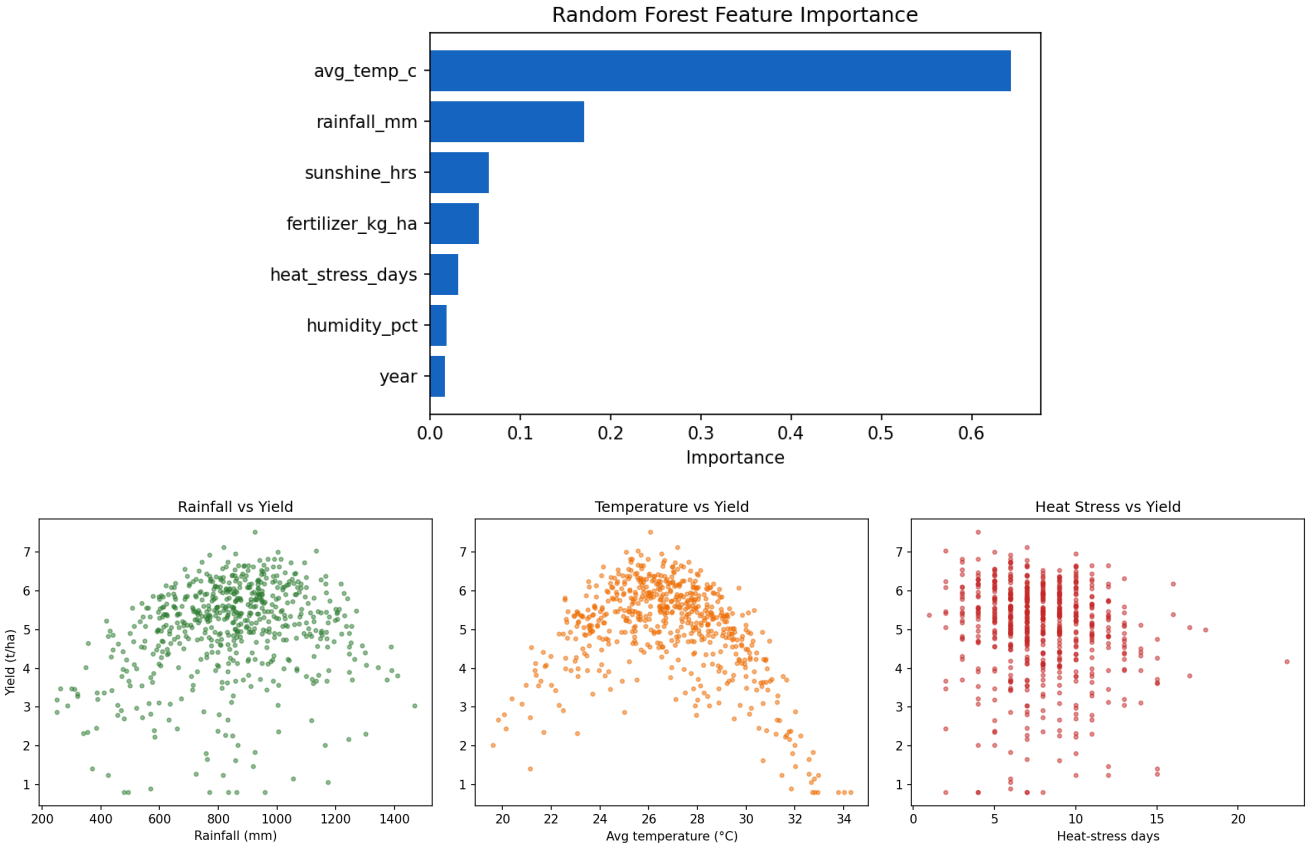
# optimum rainfall / temperature bins
rb = df.groupby(pd.cut(df.rainfall_mm, [0, 500, 700, 900, 1100, 1300, 2000]), observed=True).yield_t_ha.mean()
tb = df.groupby(pd.cut(df.avg_temp_c, [0, 23, 25, 27, 29, 31, 40]), observed=True).yield_t_ha.mean()
reg = df.groupby("region").yield_t_ha.mean().round(2)

out = {"rows": len(df), "missing_before": missing_before,
       "yield_mean": df.yield_t_ha.mean(), "yield_std": df.yield_t_ha.std(),
       "yield_min": df.yield_t_ha.min(), "yield_max": df.yield_t_ha.max(),
       "corr": corr.round(3).to_dict(), "models": res,
       "importance": imp.round(3).to_dict(),
       "rain_bins": {str(k): round(v, 2) for k, v in rb.items()},
       "temp_bins": {str(k): round(v, 2) for k, v in tb.items()},
       "region": reg.to_dict(),
       "hot_vs_normal": [df[df.heat_stress_days >= 12].yield_t_ha.mean(),
                         df[df.heat_stress_days <= 5].yield_t_ha.mean()]}
json.dump(out, open("results.json", "w"), indent=2, default=float)
print(json.dumps(out, indent=2, default=float))
```

4. Key Findings and Insights

Model	Test R2	MAE	RMSE	CV R2
Linear Regression	0.37	0.75	0.93	0.24
Random Forest	0.88	0.33	0.41	0.85

Temperature is the top driver (64% importance), then rainfall (17%). Best yield at 25-27 C and 900-1100 mm. 12+ heat-stress days cut yield by about 0.7 t/ha.



5. Conclusion

Weather affects yield non-linearly; Random Forest predicts well (R2 0.88). Focus on heat tolerance and keeping water supply near 900-1100 mm. Re-run on real data before relying on the exact optimum values.

Coding: run the project

```
pip install pandas numpy scikit-learn matplotlib
python crop_weather_analysis.py
```